

SONGDO EDGE INTELLIGENCE

송도 엣지 인텔리전스: 신뢰성 기반 로컬 분산 컴퓨팅

Student Implementation Handbook & Systems Engineering Manual

A Seven-Week Project-Based Curriculum for Operating Systems (운영체제)

Course: Operating System (운영체제)

Instructor: Prof. Mehdi Pirahandeh

Institution: Inha University, Department of Electronic Engineering

Project Mission: "Make Songdo compute locally and reliably."

Signature Demonstration: "Disconnect Internet → system continues operating."

Downstream Integration: Songdo Sensor Grid, Urban Data Platform, VIP, Smart Mobility

The Conceptual Physical-to-Process Chain:

Sensor $\xrightarrow{\text{Data}}$ Linux Edge Device $\xrightarrow{\text{Deploy}}$ Processes $\xrightarrow{\text{IPC}}$ Bounded Queues / Sockets $\xrightarrow{\text{Deploy}}$ Containers
 $\xrightarrow{\text{Compute}}$ Local Rule / Compact AI $\xrightarrow{\text{Action}}$ Simulated / Physical Actuator

Incheon Metropolitan City, Songdo International Business District
Academic Year 2026 / Spring Semester

Contents

1	Project Mission, Teams, and Boundaries	2
1.1	Project Mission: "Make Songdo compute locally and reliably."	2
1.2	The Five Engineering Teams	2
1.3	The Conceptual Chain: Data Flow vs. Deployment Boundaries	2
2	Operating Systems Foundations & Curated Command Ladder	3
2.1	Mapping Edge Features to Operating System Concepts	3
2.1.1	Virtual Memory & Bounded Buffering	3
2.1.2	Race Conditions & Critical Section Synchronization	3
2.1.3	Inter-Process Communication (IPC) Mechanisms	4
2.2	Curated Ubuntu Bash Command Ladder	4
3	The Five Team Mini-Projects & Complete 25 Scenarios	5
3.1	Team OS1: Smart Building Edge Gateway (Songdo Convensia)	5
3.2	Team OS2: Robot Edge Controller (Songdo Central Park)	6
3.3	Team OS3: Autonomous Mobility Edge Node (Techno-Park Intersection)	7
3.4	Team OS4: IoT Edge Gateway (Techno-Strip Tower)	7
3.5	Team OS5: Fault-Tolerant Smart-City Node (Waterfront Lake Sluice)	8
4	Seven-Week Implementation Plan & 100-Point Rubric	9
4.1	Seven-Week Implementation Schedule	9
4.2	100-Point Grading Rubric	9
5	Conclusion & Final Words to Students	10

1 Project Mission, Teams, and Boundaries

1.1 Project Mission: “Make Songdo compute locally and reliably.”

Modern smart cities are excessively dependent on centralized cloud infrastructure. When a wide-area network (WAN) fiber is severed or an upstream cloud datacenter suffers an outage, smart building air filtration stops, autonomous shuttles stall, and municipal water pumps lose coordination.

In **Operating System**, students design and deploy **Songdo Edge Intelligence** nodes. The foundational project mission is:

“Make Songdo compute locally and reliably.” (송도가 로컬에서 신뢰성 있게 연산하도록 만든다.)

The defining signature demonstration for every team is:

“Disconnect Internet → system continues operating.”
(인터넷 연결이 끊겨도 엣지 노드는 감지, IPC, 로컬 추론, 제어, 내구성 큐잉을 중단 없이 수행한다.)

1.2 The Five Engineering Teams

Students are organized into five teams (5 students per team, 7-week curriculum), each targeting a critical urban edge domain in Songdo:

Table 1: The Five Operating System Teams and Core Deliverables

Team	Mini-Project Name	Target Domain	Core Local Deliverable
OS1	Smart Building Edge Gateway	Songdo Convensia	Local HVAC CO ₂ ventilation & occupancy state synchronization
OS2	Robot Edge Controller	Central Park Pathways	Low-latency obstacle e-stop & periodic monotonic motion control
OS3	Autonomous Mobility Edge Node	Techno-Park Intersection	Bounded-latency crosswalk speed sign adjustment & transit events
OS4	IoT Edge Gateway	Techno-Strip Tower	Multi-sensor aggregation, PM2.5 mist suppression & cron maintenance
OS5	Fault-Tolerant Smart-City Node	Waterfront Canal Sluice	Process crash recovery, dead-lock detection & flood gate relief

1.3 The Conceptual Chain: Data Flow vs. Deployment Boundaries

Understanding the distinction between **data flow** and **deployment boundaries** is paramount:

1. **Data Flow Arrows** ($\xrightarrow{\text{Data}}$): Represent actual information exchange, such as sensor voltages converted to bytes, IPC messages passed across sockets or pipes, and actuation commands dispatched over I/O registers.
2. **Deployment / Isolation Boundaries** ($\xrightarrow{\text{Deploy}}$): Represent structural execution containers. For example, a **container** (Docker / OCI) does not perform extra mathematical signal processing; it is merely a Linux cgroup and namespace boundary wrapping one or more **processes**.

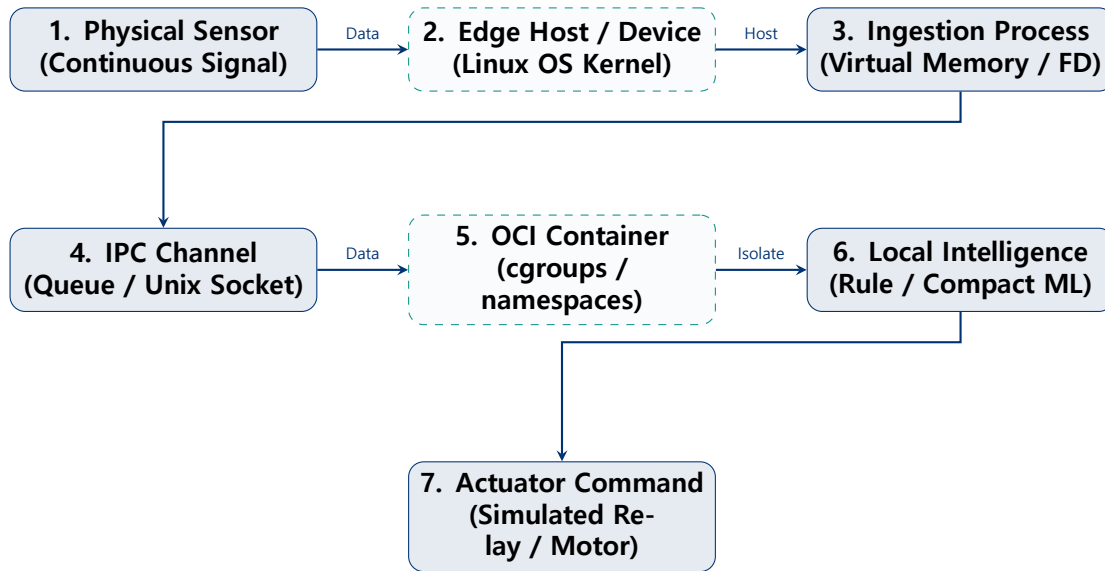


Figure 1: The Complete Conceptual Chain: Distinguishing Data Flows from Deployment Boundaries.

2 Operating Systems Foundations & Curated Command Ladder

2.1 Mapping Edge Features to Operating System Concepts

Operating System Concept: Process vs. Thread vs. systemd Service

- **Program:** An inert binary or script stored on disk ('ELF' executable or Python source).
- **Process:** An active instance of a program executing in its own private virtual address space, containing code, data, heap, stack, file descriptor table, and PID.
- **Thread:** The schedulable unit of CPU execution within a process, sharing address space and open file descriptors with sibling threads.
- **systemd Service:** A managed cgroup unit supervised by the Linux init system ('PID 1'), with automated restart policies, environment files, and resource accounting.

2.1.1 Virtual Memory & Bounded Buffering

In Python edge processing, unbounded queues cause uncontrolled memory allocation, eventually triggering the Linux Out-Of-Memory (OOM) killer. By enforcing a ****Bounded Queue**** ($N_{\max} = 500$) with a declared 'drop_{oldest}' policy, the Resident Set Size (RSS) remains strictly bounded within the allowed memory ($512M$).

2.1.2 Race Conditions & Critical Section Synchronization

When two concurrent processes increment a shared integer variable:

$$\text{counter} = \text{counter} + 1 \quad (1)$$

the operation is not atomic at the machine level. It involves three discrete CPU instructions:

$$\text{LOAD } R_0 \leftarrow \text{counter} \quad \longrightarrow \quad \text{ADD } R_0, 1 \quad \longrightarrow \quad \text{STORE counter} \leftarrow R_0$$

If an involuntary context switch occurs between LOAD and STORE, updates are lost. Process-level Mutexes ('multiprocessing.Lock') enforce mutual exclusion over the critical section.

2.1.3 Inter-Process Communication (IPC) Mechanisms

Table 2: Comparison of Linux IPC Mechanisms in Edge Nodes

IPC Mechanism	Data Structure	Kernel Crossing	Edge Role in Songdo Grid
Anonymous Pipe	Byte Stream	Yes	Unidirectional streaming parent-child
Unix Domain Socket	Stream / Datagram	Yes	Framed bidirection request-response, local network
POSIX Shared Memory	Raw Memory Buffer	No (after setup)	High-throughput telemetry (< 10 μ s latency)
Bounded Queue	Serialized Objects	Yes (via pipe)	Thread/process safe task dispatching

2.2 Curated Ubuntu Bash Command Ladder

1. Process Inspection & Hierarchy:

```

1 # Inspect process hierarchy with PIDs and command lines
2 pstree -p -a $(pgrep -f "python.*supervisor")
3
4 # Query process CPU, RSS memory, and voluntary context switches
5 ps -p <PID> -o pid,ppid,user,%cpu,rss,time,stat,wchan:20

```

2. Socket & IPC Inspection:

```

1 # List listening Unix Domain Sockets and active connections
2 ss -xlp | grep "songdo"
3
4 # Verify POSIX shared memory segments
5 ls -lh /dev/shm/

```

3. systemd Unit Management & Journal Logging:

```

1 # Check user-service status and restart counters
2 systemctl --user status songdo-edge@OS1.service
3
4 # Stream service logs with microsecond UTC timestamps
5 journalctl --user-unit songdo-edge@OS1.service -f -o short-iso

```

4. Resource Monitoring & Context Switch Profiling:

```

1 # Monitor system context switches and run-queue length
2 vmstat 1 5
3
4 # Profile per-process CPU and voluntary/involuntary context switches
5 pidstat -w -u -p <PID> 1 3

```

5. Safe Lock-Protected Shell Operations:

```

1 # Prevent overlapping cron runs using flock file descriptor lock
2 flock -n /tmp/songdo_backup.lock -c "./scripts/backup.sh"

```

3 The Five Team Mini-Projects & Complete 25 Scenarios

Every team implements five rigorous scenarios satisfying the mandatory contract across processes, IPC, persistence, scheduling, and offline survivability.

3.1 Team OS1: Smart Building Edge Gateway (Songdo Convensia)

Node ID: EDGE_NODE_OS1_CONVENSIA (Lat: 37.3888, Lon: 126.6534)

Scenario OS1-1: Local Room Sensor Ingestion & Cached Inventory

Mission & OS Target: Ingesting indoor environmental sensors and mapping against VWorld building geometry.

- **Processes:** Sensor acquisition process reading indoor CO₂ (ppm) and room temperature (°C).
- **OS Concept:** File descriptors, parsing cached VWorld building GeoJSON context, and populating SQLite 'sensor_{events}'.

Scenario OS1-2: Competing Occupancy Writers & Race Condition Demonstration

Mission & OS Target: Proving and preventing race conditions during concurrent turnstile counts.

- **OS Concept:** Demonstrating counter corruption under preemption, followed by strict Mutex lock protection.
- **Test:** Run 80 concurrent increments. Verify unprotected counter < 80, mutex counter $\equiv$ 80.

Scenario OS1-3: Local Ventilation Rule & Damper Actuator Dispatch

Mission & OS Target: Demand-controlled ventilation adjustment with command expiry TTL.

- **Rule:** If CO₂ > 1000 ppm, dispatch HVAC_DAMPER_PCT = 100%. Expiry TTL set to 15.0 s.
- **OS Concept:** Monotonic clock comparisons prevent stale command actuation.

Scenario OS1-4: Contention Profiling & Resource Sampling

Mission & OS Target: Measuring RSS and context switches under synthetic load.

- **OS Concept:** Sampling `/proc/[pid]/stat` and recording to `'processhealth'tableevery2.0 s`.

Scenario OS1-5: Signature WAN Outage & Offline Outbox Queueing

Mission & OS Target: Simulating WAN drop and verifying zero data loss during disconnection.

- **Demonstration:** Disconnect mock WAN upstream for 10 minutes. Observe events accumulating in 'durable_{outbox}'. Reconnect and verify complete batch drainage without duplicate entries.

3.2 Team OS2: Robot Edge Controller (Songdo Central Park)

Node ID: EDGE_NODE_OS2_ROBOT (Lat: 37.3925, Lon: 126.6360)

Scenario OS2-1: Multi-Process Robot Architecture & IPC Topology

Mission & OS Target: Separating LiDAR, controller, and safety monitor into distinct Linux processes.

- **OS Concept:** Unix Domain Socket and Bounded Queue IPC linking sensor and motion controllers.

Scenario OS2-2: Periodic Monotonic Deadlines & Jitter Measurement

Mission & OS Target: Measuring control loop scheduling jitter under Linux CFS.

- **Math:** Target loop period $T = 50$ ms. Measure jitter $J[n] = |\Delta t[n] - T|$.

Scenario OS2-3: Obstacle Proximity E-Stop & Actuator Command Expiry

Mission & OS Target: Immediate emergency stop triggering when LiDAR proximity < 0.8 m.

- **OS Concept:** Command validity TTL set to 500 ms; expired commands are rejected by safety latch.

Scenario OS2-4: Worker Crash & Supervised Automatic Restart

Mission & OS Target: Injecting SIGKILL into motion worker and verifying bounded recovery.

- **OS Concept:** Supervisor traps 'SIGCHLD', restarts failed child within 2.0 s, and restores state.

Scenario OS2-5: Autonomous Offline Pathway Navigation

Mission & OS Target: Continuing local navigation and logging outbox telemetry during WAN severed state.

- **Demonstration:** Zero interruption to obstacle avoidance when Internet route is disabled.

3.3 Team OS3: Autonomous Mobility Edge Node (Techno-Park Intersection)

Node ID: EDGE_NODE_OS3_MOBILITY (Lat: 37.3820, Lon: 126.6560)

Scenario OS3-1: Incheon Bus Stop Context & Vehicle Influx Stream

Mission & OS Target: Ingesting public transport sequences and vehicle speed telemetry.

- **OS Concept:** Loading cached Incheon BIS route data and correlating with crosswalk pedestrian counts.

Scenario OS3-2: Priority-Aware Bounded Queue Processing

Mission & OS Target: Prioritizing pedestrian surge alarms over background sensor logging.

- **OS Concept:** Bounded queue prevents buffer bloat and guarantees low-latency processing.

Scenario OS3-3: Overload Experiment & Drop Policy Verification

Mission & OS Target: Evaluating queue drop-oldest behavior under high event burst.

- **Result:** Queue size stays capped at 500; oldest items are discarded cleanly without crash.

Scenario OS3-4: Transactional State Update & Version Guarding

Mission & OS Target: Atomic state transitions guarded by SQLite WAL transactions.

- **OS Concept:** ACID compliance and version increments prevent torn reads across processes.

Scenario OS3-5: Offline Variable Speed Sign Dispatch & Event Replay

Mission & OS Target: Adjusting speed limit to 30 km/h during surge, queueing history for VIP ingestion.

- **Test:** Complete event conservation verified upon WAN reconnect.

3.4 Team OS4: IoT Edge Gateway (Techno-Strip Tower)

Node ID: EDGE_NODE_OS4_IOT_HUB (Lat: 37.3812, Lon: 126.6620)

Scenario OS4-1: Heterogeneous Multi-Stream Sensor Ingestion

Mission & OS Target: Aggregating PM2.5, ambient noise, and illumination across asynchronous rates.

- **OS Concept:** Polling and non-blocking I/O across diverse sensor channels.

Scenario OS4-2: Event-Time vs. Receipt-Time Handling

Mission & OS Target: Resolving out-of-order delivery from delayed sensor nodes.

- **OS Concept:** Preserving 'observed_at'eventtimestampsindependentlyofsystem'received_at'.

Scenario OS4-3: Dust Suppression Mist Cannon Relay Actuation

Mission & OS Target: Activating high-pressure mist cannon when PM2.5 exceeds Korea Bad threshold ($> 50 \mu\text{g}/\text{m}^3$).

- **Action:** Dispatches MIST_CANNON_RELAY with 30-second TTL.

Scenario OS4-4: Scheduled Database Backup via systemd Timer / Cron

Mission & OS Target: Automating online database backup with flock overlap prevention.

- **OS Concept:** Safe execution using SQLite online backup API; zero corrupt locks.

Scenario OS4-5: WAN Outage Backlog Drain & AirKorea Baseline Benchmark

Mission & OS Target: Draining queued particulate telemetry and cross-checking against Songdo Station 131233.

- **Verification:** 100% delivery acknowledgement to downstream Urban Data Platform.

3.5 Team OS5: Fault-Tolerant Smart-City Node (Waterfront Lake Sluice)

Node ID: EDGE_NODE_OS5_RELIABLE (Lat: 37.3980, Lon: 126.6210)

Scenario OS5-1: Health Check Architecture & Readiness State Machine

Mission & OS Target: Implementing liveness vs readiness probes for critical flood infrastructure.

- **OS Concept:** Periodic heartbeat reporting to 'process_{health}'table.

Scenario OS5-2: Disposable Worker Kill & Preserved State Recovery

Mission & OS Target: Evaluating supervisor response to arbitrary worker termination.

- **Test:** Kill processor worker via SIGKILL; verify supervisor respawns process without data loss.

Scenario OS5-3: Deadlock Detection & Mutex Timeout Recovery

Mission & OS Target: Preventing permanent hang during inverted resource acquisition.

- **OS Concept:** Bounded lock acquisition ('acquire(timeout=0.2)') recovers node gracefully.

Scenario OS5-4: Disaster Flood Relief Sluice Gate Actuation

Mission & OS Target: Opening backup sluice gate when canal water level exceeds critical 2.8 m threshold.

- **Action:** Dispatches BACKUP_SLUICE_GATE_OPEN to prevent urban inundation.

Scenario OS5-5: Compound Failure Survivability (WAN Cut + Worker Crash)

Mission & OS Target: Simultaneous network disconnection and worker crash resilience.

- **Result:** Worker recovers locally; outbox preserves flood events; WAN drains cleanly upon link recovery.

4 Seven-Week Implementation Plan & 100-Point Rubric

4.1 Seven-Week Implementation Schedule

Table 3: Seven-Week Implementation Roadmap for Operating Systems

Week	Topic & Lab Focus	Deliverables & Exit Gates	Weight
W1	Linux Platform, Git & Architecture	Virtualenv setup, preflight audit script, process skeleton	10%
W2	Multi-Process IPC & Bounded Queues	Bounded queue implementation, drop policy, race condition test	15%
W3	SQLite Persistence & Durable Outbox	SQLite WAL setup, atomic batch insert, outbox table schema	15%
W4	Local Decisions & Actuator Freshness	Decision engine, rule evaluation, monotonic TTL enforcement	20%
W5	Supervision, systemd & Timers	systemd unit template, automated restarts, flock backup job	15%
W6	Signature 10-Minute Offline Demonstration	WAN outage test, outbox accumulation, reconnect batch drain	15%
W7	Final Integration, Defense & Release	25-scenario execution, individual oral defense, technical report	10%

4.2 100-Point Grading Rubric

1. Operating Systems Core Concepts & IPC Architecture (20 Points):

- Clean multi-process architecture with distinct roles (Sensor, Processor, Health, UI).
- Proper use of bounded queues, Unix domain sockets, and IPC shutdown sentinels.
- Elimination of zombie processes via proper signal trapping and 'wait()' joins.

2. Synchronization & Data Durability Correctness (15 Points):

- Empirical race condition reproduction followed by strict Mutex resolution.
- Deadlock prevention via lock ordering or timeout-based recovery.
- SQLite WAL mode configuration with busy timeout and uncorrupted online backup.

3. Executable Python/Bash Implementation & Local Processing (15 Points):

- Working Bash ladder scripts ('preflight.sh', 'setup.sh', 'start_{demo}.sh').
- Rule-based edge decision logic with honest labeling and compact inference.
- Actuator command freshness checking via monotonic clock comparisons.

4. Supervision, Scheduling & Containerization (15 Points):

- Functional systemd user service unit with burst limits and resource constraints.
- Flocked scheduled maintenance script preventing overlapping runs.
- Valid OCI Dockerfile and compose configuration with healthcheck.

5. Signature Offline Demonstration & Event Conservation (10 Points):

- 10-minute WAN disconnect test: zero crash, continuous local actuation, zero lost events.
- Outbox reconnect batch drainage with idempotent receiver deduplication.

6. Local Web Console Usability & Telemetry Exposure (5 Points):

- Functional zero-dependency local dashboard displaying live PIDs, CPU/RAM, and out-box.

7. Individual Contribution & Oral Technical Defense (20 Points):

- Individual mastery of virtual memory, context switching, locks, and signal handling.

5 Conclusion & Final Words to Students

Real-world systems engineering is defined by how software behaves when conditions are adverse. By building an edge node that continues operating seamlessly when the Internet disappears, you master the true power of the operating system.

Prof. Mehdi Pirahandeh

Inha University, Department of Electronic Engineering
Songdo International Business District, 2026